

Coffee Run

A privacy-first macOS menu bar app for spontaneous office coffee runs.
Discovers nearby coworkers over Bonjour, no cloud accounts, no servers.

Repo: github.com/rutasraju/coffee-run

Bundle ID: com.ruta.coffeebreak

Platform: macOS 13+ (Ventura and newer)

Language: Swift 5.9 / SwiftUI + AppKit

Distribution: Ad-hoc signed DMG · App Store ready

Contents

- | | |
|----------------------------------|------------------------------------|
| 1. Product overview | 9. Invite a coworker (QR & share) |
| 2. User-facing features | 10. Global hotkey <code>^⌘C</code> |
| 3. Architecture at a glance | 11. Demo mode |
| 4. Networking (Bonjour) | 12. Steam animations |
| 5. State management | 13. Privacy model |
| 6. Persistence & iCloud sync | 14. File map |
| 7. Notifications & rate-limiting | 15. Build, sign & distribute |
| 8. Caffeinated Mode (power) | 16. Roadmap |

1. Product overview

Coffee Run is a **menu bar app** that lets a small group of coworkers discover each other's coffee intent without anyone opening Slack, sending a message, or syncing calendars. Click the cup in the menu bar, declare "I want coffee" or start a Coffee Run, and anyone nearby running the app sees it.

The product philosophy is intentionally narrow:

- **Local network only.** Bonjour / mDNS discovers peers on the same Wi-Fi. Nothing leaves the local subnet.
- **No accounts, no servers.** No login, no analytics. Identity is a device-local UUID.
- **Ambient, not chatty.** The menu shows passive presence — like a hallway. Active "buzz" notifications are rate-limited and easily silenced.
- **Spontaneity over scheduling.** Runs default to *now*. "In 5" / "In 15" exist but aren't the main path.

Why menu bar? Coffee runs are decided in 5 seconds, not 5 minutes. A full app window would lose against the friction of opening Slack. The menu bar is the only surface that survives the comparison.

2. User-facing features

2.1 The menu

Single popover anchored on the menu bar cup icon. Width 360pt, dynamic height 720–1200pt. Layout, top to bottom:

- **Header bar** — "Coffee Run" title + ⚙ Settings icon.
- **Buzz permission banner** — only shown when macOS notifications are denied or not-yet-determined.

- **Hero status card** — illustrated cup + animated steam + dynamic headline ("Brewing with the X crew" / "Looking for coffee company") + breathing status dot.
- **Presence chip** — stacked avatars + warm prose like "Fresh coffee energy nearby" / "Just you for now · Invite a coworker to fill it up".
- **Stats row** — two tiles: *Cups brewing with your crew* (today) and *Coffees sparked by you this week* (with streak).
- **Crew filter row** — Picker: "Everyone nearby" or a specific crew.
- **Three big action buttons** — Coffee Run · Available · Heads Down.
- **Run options panel** — appears when Coffee Run is tapped: audience chips + timing chips (Now / In 5 / In 15).
- **Incoming crew invites** — Accept / Decline cards (broadcast over Bonjour).
- **What's happening nearby** — live cards for each active run with joiners and a Join button. Empty state explains the silence.
- **Utility rows** — Caffeinated Mode · Mute for 1 hour · Add note · Invite a coworker · My coffee energy.
- **Bottom section** — Settings · Profile · Quit.

2.2 Coffee status

Status	Symbol	What others see	Auto-expires?
Available	cup outline	"X is around"	No
Heads Down	cup outline + red ×	You don't appear	No
I want coffee	filled cup	"X wants coffee"	Yes (default 30 min)
Coffee Run (goingNow)	filled cup + green →	"X started a coffee run"	Yes (auto-logs as cup)
Joining	filled cup + blue 🙌	"X joined Y's run"	Yes

Passive choices (Available / Heads Down) persist across launches. Coffee signals always expire back to the last passive state — Coffee Run never sticks you in "wanting coffee" forever.

2.3 Coffee runs

A run = one organizer + zero or more joiners, all broadcasting the same `runID` over Bonjour TXT records. Runs can be:

- **Open** — visible to everyone on the local network.
- **Audience-scoped** — broadcast a group ID; only members of that group render the run in their menu.
- **Scheduled** — `startsAt` in the TXT record drives "Leaving in 5 min" labels.

When a run expires without being explicitly ended, the app auto-logs one cup for the organizer and any joiners. Logging a cup mid-run ends your participation early and bumps your weekly tally.

2.4 Crews (groups)

Lightweight group abstraction for offices bigger than a small team. Create a crew (Profile & Settings → Crews), invite specific peers, and they receive a system notification + an Accept/Decline card in their menu. Crews are local — joining is a two-party handshake, not a server-mediated membership list.

2.5 Stats & coffee energy

Two values surfaced in the menu:

- **Cups brewing with your crew (today)** — sum of your cups + every visible peer's cups today, broadcast in the TXT record.
- **Coffees sparked by you this week** — your own count, with a fire-emoji day streak when consecutive days have at least one cup.

Profile & Settings → Profile shows lifetime cups, days active, and the count of people you've ever brewed coffee with nearby.

2.6 Caffeinated Mode

Prevents the display from sleeping using an `IOPMAssertion` (type `kIOPMAssertionTypeNoDisplaySleep`). Pure power feature — does not change your coffee status. Indicator: a yellow ⚡ bolt next to the cup in the menu bar.

2.7 Buzzes (notifications)

"Buzz" is the product term for a system notification. Two categories registered with `UNUserNotificationCenter`:

- **COFFEE_CATEGORY** — actions: Join · Dismiss. Fired when a nearby peer starts a coffee signal.
- **GROUP_INVITE_CATEGORY** — actions: Accept · Decline. Fired when a crew invite targeting you arrives.

Buzzes are rate-limited to **3 per 10-minute window**, dropped while muted, and can be silenced for 1 hour from a menu row. Custom `coffee.caf` sound bundled in `Resources/`.

2.8 Invite a coworker

Top-level menu row that opens a dedicated window with:

- A high-resolution QR code (CIFilter, 10× oversampled) pointing to the GitHub repo.
- The link spelled out in monospaced font for read-aloud / typed sharing.
- **Copy link** — writes the URL to the pasteboard with a checkmark confirmation.
- **Share...** — opens `NSSharingServicePicker` for AirDrop, Mail, Messages, etc.

2.9 Global hotkey `^⌘C`

Press `Control` + `Option` + `C` anywhere on macOS to start an immediate Coffee Run open to everyone — the menu pops so you can adjust or cancel. Implemented with paired global + local `NSEvent` monitors; no Accessibility permission required.

2.10 Demo mode

Settings → General → "Try it out" — toggles on a stable fake crew (Maya organizing a run starting in 2 min, Dev joined her, Sara available, Jordan wants coffee). Useful for screenshots, demos, and brand-new users who haven't onboarded coworkers yet. Nothing is broadcast and stats don't change.

2.11 Steam animations

Three procedural steam systems built on `TimelineView` + `Canvas` :

- **SteamWisps** — small particles for the hero cup edge.
- **SteamHybrid** — slow expanding cloud body + fast detail particles. Used as the hero cup overlay.
- **SteamRibbons** — two animated bezier S-curves with time-varying control points. Used on stat tiles.

A full-screen steam overlay fires on personal coffee actions and when someone joins your run. Suppressed while Caffeinated or Muted.

3. Architecture at a glance

Process model

Single SwiftUI `App` with `@NSApplicationDelegateAdaptor`. `MenuBarExtra` renders the menu. `LSUIElement=true` + `.accessory` activation policy = no Dock icon by default.

State

Single `AppState: ObservableObject` marked `@MainActor`. Views observe via `@EnvironmentObject`. All mutations funnel through methods on this class.

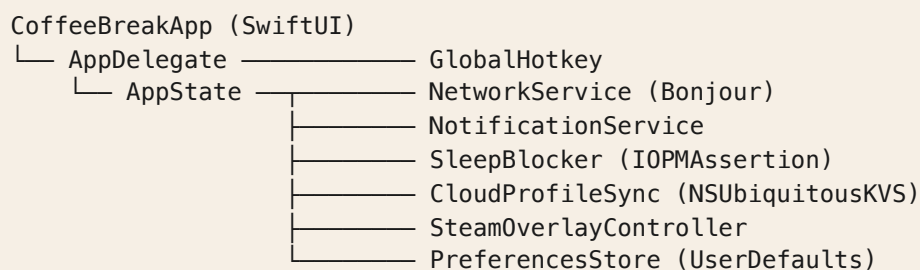
Services

Discrete classes for Bonjour, notifications, sleep, iCloud, QR, and global hotkey. Held by `AppState` or `AppDelegate`, never created ad-hoc by views.

Persistence

`UserDefaults` via `PreferencesStore` wrapper. `UserProfile` serialized as Codable JSON. Optional `NSUbiquitousKeyValueStore` for cross-Mac sync.

Dependency graph



4. Networking (Bonjour)

The whole peer-discovery story is built on Apple's `Network.framework`. Service type: `_coffeebreak._tcp`.

4.1 TXT record schema

All per-peer state is broadcast in the Bonjour TXT record. There is no separate TCP channel — peers don't talk to each other directly, they just publish state.

Key	Type	Meaning
<code>id</code>	UUID string	Stable peer ID (lives in UserDefaults)
<code>name</code>	String	Display name
<code>status</code>	enum string	available / notAvailable / wantCoffee / goingNow / joining
<code>ts</code>	Unix seconds	When this status was set
<code>note</code>	String?	Optional free-text ("kitchen", "lobby")
<code>run</code>	UUID?	Run identity — equals <code>id</code> if organizer, points elsewhere if joining
<code>cups</code>	Int	Cups logged today (resets at midnight)
<code>startsAt</code>	Unix seconds?	Future start time for scheduled runs
<code>groups</code>	comma UUIDs	Crews this peer is a member of
<code>invs</code>	encoded list	Crew invites this peer is offering
<code>aud</code>	UUID?	Audience-scoped run target group

4.2 Discovery flow

1. `NWListener` publishes our TXT record under `_coffeebreak._tcp`.
2. `NWBrowser` watches for other peers and emits results.
3. Each result's TXT is parsed into a `Peer` struct.
4. The combined peer list is published from `NetworkService.$peers` and observed by `AppState`.
5. The browse callback uses `MainActor.assumeIsolated` to keep Swift 6 strict-concurrency happy.

4.3 What goes wrong on real networks

- **VLANs:** Same Wi-Fi name does not mean same broadcast domain. Office Wi-Fi often segregates devices — Coffee Run silently shows no peers in that case.
- **Sleep / wake:** Bonjour services re-register on wake; brief gap is expected.
- **Hotel / public Wi-Fi:** Client isolation blocks peer discovery entirely.

5. State management

`AppState` is the single source of truth, marked `@MainActor`. Views never reach into services directly.

5.1 Published properties

- **Identity:** `displayName`, `hasCompletedSetup`, `ownPeerID` (let).
- **Status:** `ownStatus`, `ownNote`, `statusTimestamp`, `ownRunID`, `ownStartsAt`, `ownAudienceGroupID`.
- **Profile:** `profile` (`UserProfile` struct).
- **Preferences:** `expiryMinutes`, `notificationsEnabled`, `soundEnabled`, `startAtLogin`, `showSteamForAllSignals`, `cloudSyncEnabled`, `demoModeEnabled`.
- **Peers:** `peers`, `incomingSignalTick` (animation pulse).
- **Power & quiet:** `isCaffeinated`, `mutedUntil`.
- **UI hooks:** `activeGroupFilter`, `notificationAuthorizationStatus`.
- **Closures wired by AppDelegate:** `dismissMenuBar`, `openSettings`, `openInvite`.

5.2 Derived state

- `activePeers` — peers fresh within the expiry window, merged with demo peers when demo mode is on, filtered by `activeGroupFilter`.
- `activeRuns` — organizers visible to us (respecting audience scope), each paired with their visible joiners.
- `nearbyCoffeeSeekers`, `coffeeDrinkerCount`, `totalCoffeesOnNetwork`, `pendingIncomingInvites`.
- `menuBarSymbolName`, `statusSymbolName`, `menuBarAccent`, `menuBarBadgeCount` — drive the icon in the menu bar.

6. Persistence & iCloud sync

6.1 PreferencesStore (UserDefaults)

Thin struct wrapping `UserDefaults.standard`. Notable keys:

- `displayName`, `peerID` — identity.
- `expiryMinutes`, `notificationsEnabled`, `soundEnabled`, `startAtLogin` — prefs.
- `userProfile_v1` — JSON-encoded `UserProfile`.
- `cloudSyncEnabled`, `showSteamForAllSignals`, `demoModeEnabled`.
- `lastPassiveStatus` — the user's last `Available` / `Heads Down` choice. Coffee signals always expire to this.
- `mutedUntil` — Date. Mute survives relaunches.

6.2 UserProfile + CRDT merge

The profile is designed so two Macs syncing the same identity can converge deterministically without a coordinator.

- `coffeesByDate: [String: Int]` — per-day counts, merged by taking the max per date.
- `peopleMet: Set<UUID>` — union.
- `joinedGroups: [CoffeeGroup]` — union by group id.
- `declinedGroupInvites: Set<UUID>` — union.
- `createdAt` — minimum.
- `displayName`, `avatarEmoji` — last-writer-wins by `metadataUpdatedAt`.

Computed: `coffeesToday`, `coffeesThisWeek`, `weeklyDelta`, `coffeeStreak`, `daysActive`.

6.3 iCloud sync

`CloudProfileSync` wraps `NSUbiquitousKeyValueStore`. Saves on every profile mutation when enabled, listens for `didChangeExternallyNotification` to re-merge remote changes. Requires Developer ID signing to activate — dormant under ad-hoc signing.

7. Notifications & rate-limiting

7.1 Categories

```
COFFEE_CATEGORY      → [Join, Dismiss]
GROUP_INVITE_CATEGORY → [Accept, Decline]
```

7.2 Delegate routing

`NotificationService` conforms to `UNUserNotificationCenterDelegate`. The `didReceive` response handler dispatches on (category, action) and invokes one of three callbacks set by `AppState`:

- `onJoin` — flips own status to `.joining` and surfaces the menu.
- `onAcceptGroupInvite(inviterID, groupID)` — looks up the pending invite and joins the crew.
- `onDeclineGroupInvite(inviterID, groupID)` — records the decline so future broadcasts of the same invite are dropped.

7.3 Rate-limiting

1. **Mute** — when `mutedUntil > now`, all buzzes are dropped silently. Also kills full-screen steam.
2. **Window cap** — at most `maxNotificationsPerWindow = 3` per `notificationWindowSeconds = 600` (10 min) rolling window. Beyond that, the in-app steam pulse still fires but no system buzz is posted.

8. Caffeinated Mode (power)

`SleepBlocker` wraps a single `IOPMAssertion` :

```
IOPMAssertionCreateWithName(  
    kIOPMAssertionTypeNoDisplaySleep,  
    IOPMAssertionLevel(kIOPMAssertionLevel0n),  
    "Coffee Run – keeping screen awake" as CFString,  
    &assertionID)
```

Release with `IOPMAssertionRelease(assertionID)` when toggled off or on app quit. The yellow ⚡ in the menu bar is rendered alongside the cup symbol via the `MenuBarExtra` label.

The utility row is tap-to-toggle the entire row: putting an interactive `Toggle` inside a `Button` swallows the tap, so the toggle inside the row is `.constant(...)` + `.allowsHitTesting(false)` — visual indicator only, the button handles the action.

9. Invite a coworker (QR & share)

9.1 Entry points

- Top-level utility row in the menu (always visible).
- Indirect: the empty "What's happening" state copy mentions "invite a coworker" so the action is discoverable.

9.2 The sheet

`InviteSheet.swift` rendered into a small dedicated `NSWindow` (340×460pt, titled + closable). Three actions:

- **QR code** — generated by `QRCodeGenerator` via `CIFilter.qrCodeGenerator()`, oversampled 10× for crispness, rendered with `.interpolation(.none)`.
- **Copy link** — writes the URL to `NSPasteboard.general`, swaps the icon to a checkmark for 1.5s.
- **Share...** — anchored `NSSharingServicePicker` for AirDrop / Mail / Messages.

9.3 Link strategy

Currently points at `github.com/rutasraju/coffee-run`. Once a GitHub Release exists, swap to `/releases/latest` so scanning the QR jumps straight to the DMG download.

10. Global hotkey `⌘⌥C`

`GlobalHotkey.swift` registers two event monitors so the hotkey works regardless of which app has focus:

- `NSEvent.addGlobalMonitorForEvents(matching: .keyDown)` — fires when another app is frontmost.

- `NSEvent.addLocalMonitorForEvents(matching: .keyDown)` — fires when Coffee Run itself is frontmost (e.g. Settings window). Returns `nil` to consume the event so AppKit doesn't beep.

Modifier check uses `event.modifierFlags.intersection(.deviceIndependentFlagsMask) == [.control, .option]`, character match is `charactersIgnoringModifiers?.lowercased() == "c"`. **No Accessibility permission required** — these monitors observe, they don't inject.

The trigger calls `appState.startCoffeeRunFromHotkey()`, which:

1. Starts an immediate Coffee Run open to everyone (`inMinutes: 0`, `audience: nil`).
2. Activates the app so the menu pops, giving the user a chance to cancel or adjust.

11. Demo mode

Off by default. When enabled (Settings → General → "Try it out"), `AppState.refreshDemoPeers()` populates four synthetic peers with stable UUIDs:

- **Maya P.** — organizing a run starting in 2 min, note "Meet at kitchen".
- **Dev R.** — already joined Maya's run.
- **Sara K.** — available, has 2 cups logged today.
- **Jordan L.** — wants coffee, note "lobby?".

Demo peers are merged into `activePeers` via a computed property — real `peers` state is untouched, nothing is broadcast, and stats don't accumulate. Stable UUIDs ensure the avatar colors (deterministic from name+id hash) don't shuffle when you toggle the mode on and off.

12. Steam animations

System	File	Used for	Cost
SteamWisps	<code>Views/SteamWisps.swift</code>	Small accents (welcome screen, fallback cup symbol)	2–3 particles
SteamHybrid	<code>Views/SteamHybrid.swift</code>	Hero cup overlay — slow expanding cloud + fast detail	5 body + 7 detail particles, blur 0.7
SteamRibbons	<code>Views/SteamRibbons.swift</code>	Stat tile cup graphics	2 bezier S-curves, gradient stroke
Full-screen steam	<code>Services/SteamOverlay...</code>	Celebration on personal action / run join	Transparent NSWindow, ~13s loop

All systems are `TimelineView`-driven, with per-particle pseudorandom seeds for phase, speed, sway, and size jitter. The combination is what makes the steam read as natural rather than mechanical.

13. Privacy model

- **No accounts.** Identity is a UUID generated on first launch and stored in `UserDefaults`.
- **No servers.** All discovery is via Bonjour on the local subnet.
- **No analytics.** Nothing is reported to any third party.
- **iCloud sync is opt-in** and only carries the local `UserProfile` (cups, people met, crews) — never live status.
- **System notifications** are local-only and respect macOS's permission model.

Marketing copy across the app reinforces this: footer reads "Only visible to your crew nearby."

14. File map

App entry

- `CoffeeBreak/CoffeeBreakApp.swift` — `@main`, `MenuBarExtra`, label closure (inlined `HStack` — nested struct labels break click handling).
- `CoffeeBreak/Info.plist` — `LSUIElement=true`, bundle metadata.
- `CoffeeBreak/CoffeeBreak.entitlements` — entitlements for distribution.

Services

- `Services/AppDelegate.swift` — owns `AppState`, settings window, invite window, global hotkey wiring.
- `Services/NetworkService.swift` — Bonjour publish + browse, TXT encode/decode.
- `Services/NotificationService.swift` — UN categories, delegate, test buzz.
- `Services/PreferencesStore.swift` — `UserDefaults` wrapper.
- `Services/SleepBlocker.swift` — `IOPMAssertion` wrapper.
- `Services/CloudProfileSync.swift` — `NSUbiquitousKeyValueStore` wrapper.
- `Services/GlobalHotkey.swift` — `^⌘C` event monitors.
- `Services/QRCodeGenerator.swift` — `CIFilter` QR helper.

Models

- `Models/Peer.swift` — over-the-wire peer struct.
- `Models/CoffeeRun.swift` — organizer + joiners.
- `Models/CoffeeGroup.swift` — crew + invite types.
- `Models/UserProfile.swift` — Codable, CRDT merge.
- `Models/CoffeeStatus.swift` — enum + symbols.

State

- `State/AppState.swift` — the brain.

Views

- `Views/MenuContentView.swift` — main menu (the big one).

- `Views/WelcomeView.swift` — first-run name entry.
- `Views/PreferencesView.swift` — settings tabs.
- `Views/GroupsSettingsView.swift` — crew management.
- `Views/RunCard.swift` — small run card variant.
- `Views/InviteSheet.swift` — QR + share window.
- `Views/SteamHybrid.swift` — hero steam.
- `Views/SteamRibbons.swift` — stat-tile steam.
- `Views/SteamWisps.swift` — small steam accents.
- `Views/ChipFlowLayout.swift` — wrapping chip layout.

Resources

- `Resources/coffee.caf` — buzz sound.
- `Resources/coffeerun_herobig.png` + `@2x` + `@3x` — hero illustration.
- `Resources/coffeerun_spark.png` + `@2x` + `@3x` — personal-tile illustration.
- `Resources/AppIcon.icns` — app icon.

15. Build, sign & distribute

15.1 SwiftPM build

```
swift build -c release
```

Output: `.build/release/CoffeeBreak` — a Mach-O executable, not a bundle.

15.2 Bundle assembly

`Coffee Run.app` is hand-assembled (no Xcode build phase). The flow:

```
cp .build/release/CoffeeBreak "Coffee Run.app/Contents/MacOS/CoffeeBreak"
codesign --force --deep --sign - "Coffee Run.app"
```

Ad-hoc signing (`--sign -`) lets the app run locally without a Developer ID. iCloud sync stays dormant under ad-hoc — needs a real team signature to activate.

15.3 DMG

```
mkdir dmg-staging
cp -R "Coffee Run.app" dmg-staging/
ln -s /Applications dmg-staging/Applications
hdiutil create -volname "Coffee Run" \
  -srcfolder dmg-staging -ov -format UDZ0 "Coffee Run.dmg"
```

Result: ~6.4 MB self-contained installer.

15.4 Distribution paths

Path	Pros	Cons
Direct DMG (current)	Zero infra, instant updates	Gatekeeper warning on first open without notarization
Notarized + Developer ID	Clean install, iCloud sync activates	Requires Apple Developer Program (\$99/yr)
Mac App Store	One-click install, auto-updates, surface area	Sandbox restrictions, review cycle

16. Roadmap

Shipped

- v1.0 — status + crews + stats + Caffeinated Mode + iCloud sync hook + full-screen steam
- v1.1 — Invite sheet (QR + share), global hotkey `^⌘C`, demo mode, onboarding empty state

Phase B — Invite (deep-link)

Replace the GitHub link with `coffeerun://join?crew=...` deep-link that auto-joins a specific crew on first launch. Requires URL scheme registration in Info.plist + handler in AppDelegate.

Phase C — Coffee roulette

Opt-in matchmaking inside crews, falling back to anyone nearby. Surfaces a daily "you and Maya haven't had coffee yet — try now?" prompt.

Phase D — Rhythm nudges

Learn the user's usual coffee times from `UserProfile.coffeesByDate` + timestamps, fire a subtle menu-bar pulse at predicted times. Optional weekly summary card.

Phase E — Calendar awareness

EventKit permission. When the user taps Join on a run and a meeting starts within 10 min, soften the button to a confirmation prompt.